

Gamblang

1. Architettura

Gamblang è un linguaggio imperativo con supporto a costrutti probabilistici.

Il compilatore traduce il codice sorgente in C e delega la compilazione finale a GCC.

Il processo è suddiviso nelle seguenti fasi:

1. Parsing del sorgente.
2. Costruzione dell'AST.
3. Analisi semantica.
4. Generazione del codice C.
5. Compilazione del file C tramite GCC.

Il compilatore è implementato in Python.

2. Parsing

Il parsing è affidato a Lark con grammatica LALR(1).

Whitespaces

Spazi, tabulazioni e ritorni a capo non hanno significato sintattico e vengono ignorati dal parser.

L'indentazione serve esclusivamente a migliorare la leggibilità del codice.

Blocchi

Le strutture che introducono un blocco (if, loop, risky) terminano con la parola chiave end.

Esempio:

```
if score > 10
    say score
end
```

Espressioni

La precedenza degli operatori è definita nella grammatica.

Moltiplicazione e divisione hanno precedenza maggiore rispetto ad addizione e sottrazione. Le parentesi possono essere utilizzate per modificare l'ordine di valutazione.

3. Analisi Semantica

L'analisi semantica viene eseguita sull'AST prima della generazione del codice.

Symbol Table

Il compilatore mantiene una tabella dei simboli contenente le variabili dichiarate e il relativo tipo.

I tipi primitivi supportati sono:

- STEADY
- COIN
- GAMBLE

Durante questa fase vengono rilevati:

- utilizzo di variabili non dichiarate;
- dichiarazioni duplicate;
- assegnazioni incompatibili con il tipo della variabile.

Gestione delle variabili GAMBLE

Per ogni variabile GAMBLE viene mantenuto un contatore delle assegnazioni effettuate tramite l'operatore <-.

Questa informazione viene utilizzata per verificare che alcune operazioni possano essere eseguite in modo valido.

Regole di assegnazione

Le variabili STEADY utilizzano l'operatore =.

Le variabili GAMBLE utilizzano l'operatore <-.

L'utilizzo dell'operatore errato genera un errore di compilazione.

Funzione ask

La funzione ask consente di acquisire input da tastiera.

Non può essere utilizzata su variabili COIN.

Se applicata a una variabile GAMBLE, il compilatore verifica che la variabile abbia già ricevuto almeno un'assegnazione tramite <-.

4. Generazione del Codice

Il backend converte l'AST in codice C.

Ogni nodo dell'albero sintattico viene tradotto nel corrispondente costruito C.

Mappatura dei tipi

I valori numerici e booleani vengono rappresentati tramite il tipo C:

double

Questa scelta uniforma la rappresentazione interna dei dati ed evita conversioni tra tipi numerici differenti.

Tipo GAMBLE

Le variabili GAMBLE vengono rappresentate tramite una struttura dedicata:

```
typedef struct {  
    double history[100];  
    int count;  
} gamble_t;
```

history contiene i valori assegnati alla variabile.

count indica il numero di valori memorizzati.

Ogni assegnazione tramite <- aggiunge un nuovo elemento allo storico.

Quando una variabile GAMBLE viene utilizzata in un'espressione, il valore restituito viene selezionato casualmente tra quelli presenti nello storico.

La lettura viene tradotta nel seguente accesso:

```
var.history[rand() % var.count]
```

5. Runtime Probabilistico

Le funzionalità probabilistiche del linguaggio sono implementate tramite il generatore pseudo-casuale della libreria standard C.

All'inizio della funzione main viene eseguita l'inizializzazione:

```
srand(time(NULL));
```

I confronti probabilistici utilizzano valori generati tramite:

```
(double)rand() / RAND_MAX
```

Il risultato appartiene all'intervallo [0.0, 1.0].

Tipo COIN

Una variabile COIN rappresenta una probabilità.

Durante la valutazione viene confrontata con un valore casuale generato a runtime.

Blocco risky

Il blocco risky esegue il proprio contenuto con una probabilità specificata dall'espressione associata.

Esempio:

```
risky 0.25
  say "success"
end
```

In questo caso il blocco viene eseguito con probabilità pari al 25%.

Normalizzazione delle probabilità

Le espressioni probabilistiche vengono limitate all'intervallo [0.0, 1.0].

Il codice generato include la macro:

```
#define CLAMP_PROB(p) \
  ((p) > 1.0 ? 1.0 : ((p) < 0.0 ? 0.0 : (p)))
```

La macro viene applicata ai valori utilizzati dai costrutti probabilistici prima della loro valutazione.

6. Output Generato

Il compilatore produce un file C completo e compilabile.

Il file contiene:

- dichiarazioni dei tipi runtime;
- funzioni di supporto;
- traduzione delle variabili definite nel sorgente;
- traduzione delle strutture di controllo;
- funzione main.

.